

CSE 385: Analysis of Algorithms (Honors)

Lecture 5 (Divide-and-Conquer Algorithms: Integer Multiplication)

**Rezaul A. Chowdhury
Department of Computer Science
SUNY Stony Brook
Spring 2026**

Multiplying Two n -bit Numbers

$$\begin{array}{l}
 x = \overbrace{\boxed{x_L} \boxed{x_R}}^{\substack{\frac{n}{2} \text{ bits} \quad \frac{n}{2} \text{ bits}}} = 2^{n/2}x_L + x_R \\
 y = \underbrace{\boxed{y_L} \boxed{y_R}}_{n \text{ bits}} = 2^{n/2}y_L + y_R
 \end{array}$$

$$xy = (2^{n/2}x_L + x_R)(2^{n/2}y_L + y_R) = 2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R$$

So # $\frac{n}{2}$ -bit products: 4

bit shifts (by n or $\frac{n}{2}$ bits): 2

additions (at most $2n$ bits long) : 3

We can compute the $\frac{n}{2}$ -bit products recursively.

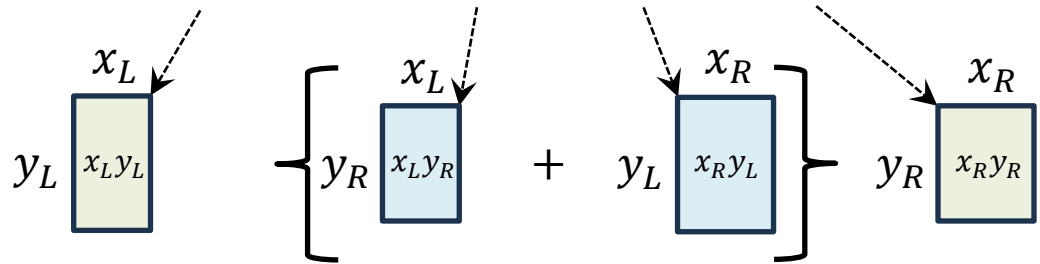
Let $T(n)$ be the overall running time for n -bit inputs.

$$\text{Then } T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 4T\left(\frac{n}{2}\right) + \Theta(n) & \text{otherwise.} \end{cases} = \Theta(n^2) \quad (\text{how?})$$

Multiplying Two n -bit Numbers

$$\begin{aligned}
 x &= \overbrace{\begin{bmatrix} x_L & x_R \end{bmatrix}}^{\frac{n}{2} \text{ bits}} = 2^{n/2}x_L + x_R \\
 y &= \underbrace{\begin{bmatrix} y_L & y_R \end{bmatrix}}_{n \text{ bits}} = 2^{n/2}y_L + y_R
 \end{aligned}$$

$$xy = (2^{n/2}x_L + x_R)(2^{n/2}y_L + y_R) = 2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R$$



Observation:

$$\begin{aligned}
 & y_L \begin{bmatrix} x_L \\ x_L y_L \end{bmatrix} + y_L \begin{bmatrix} x_R \\ x_R y_L \end{bmatrix} + y_R \begin{bmatrix} x_L \\ x_L y_R \end{bmatrix} + y_R \begin{bmatrix} x_R \\ x_R y_R \end{bmatrix} = \begin{matrix} x_L + x_R \\ y_L \begin{bmatrix} x_L y_L & x_R y_L \\ x_L y_R & x_R y_R \end{bmatrix} \\ y_R \end{matrix}
 \end{aligned}$$

$$\Rightarrow y_L \begin{bmatrix} x_R \\ x_R y_L \end{bmatrix} + y_R \begin{bmatrix} x_L \\ x_L y_R \end{bmatrix} = \begin{matrix} x_L + x_R \\ y_L \begin{bmatrix} x_L y_L & x_R y_L \\ x_L y_R & x_R y_R \end{bmatrix} \\ y_R \end{matrix} - y_L \begin{bmatrix} x_L \\ x_L y_L \end{bmatrix} - y_R \begin{bmatrix} x_R \\ x_R y_R \end{bmatrix}$$

Multiplying Two n -bit Numbers Faster (Karatsuba's Algorithm)

$$\begin{array}{l}
 x = \overbrace{\boxed{x_L} \boxed{x_R}}^{\substack{\frac{n}{2} \text{ bits} \quad \frac{n}{2} \text{ bits}}} = 2^{n/2}x_L + x_R \\
 y = \underbrace{\boxed{y_L} \boxed{y_R}}_{n \text{ bits}} = 2^{n/2}y_L + y_R
 \end{array}$$

$$\begin{aligned}
 xy &= (2^{n/2}x_L + x_R)(2^{n/2}y_L + y_R) \\
 &= 2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R \\
 &= 2^n x_L y_L + 2^{n/2}((x_L + x_R)(y_L + y_R) - x_L y_L - x_R y_R) + x_R y_R
 \end{aligned}$$

So # $\frac{n}{2}$ - or $(\frac{n}{2} + 1)$ -bit products: 3

Then the overall running time for n -bit inputs:

$$\begin{aligned}
 T(n) &= \begin{cases} \Theta(1) & \text{if } n = 1, \\ 3T\left(\frac{n}{2}\right) + \Theta(n) & \text{otherwise.} \end{cases} \\
 &= \Theta(n^{\log_2 3}) = O(n^{1.59}) \text{ (how? derive)}
 \end{aligned}$$

Algorithms for Multiplying Two n -bit Numbers

Inventor	Year	Complexity
Classical	—	$\Theta(n^2)$
Anatolii Karatsuba	1960	$\Theta(n^{\log_2 3})$
Andrei Toom & Stephen Cook (generalization of Karatsuba's algorithm)	1963 – 66	$\Theta \left(n 2^{\sqrt{2 \log_2 n}} \log n \right)$
Arnold Schönhage & Volker Strassen (Fast Fourier Transform)	1971	$\Theta(n \log n \log \log n)$
Martin Fürer (Fast Fourier Transform)	2005	$n \log n 2^{O(\log^* n)}$
David Harvey & Joris van der Hoeven (Fast Fourier Transform)	2019	$O(n \log n)$

Lower bound: $\Omega(n)$ (why?)

*When the math
gets tough*

*Analysis of Algorithms
in
Rhymes and Pictures*

ChatGPT 4o mini



Karatsuba's Algorithm: A Rhyme

When numbers get large and time runs tight,
Karatsuba steps in to do it right!
It breaks big tasks into pieces three,
And multiplies faster—just wait and see!

Step 1: Split the numbers—let's begin,
Each one halves, both thick and thin.
Now $x = a.b$ is split in two,
And $y = c.d$ is the other one for you!

Step 2: Multiply the parts, not four but three,
Here's the magic, as you'll see:
 ac , bd , and $(a + b)(c + d)$ —that's the key,
These three products are all you need to be.

Step 3: Now combine them with a clever blend,
Subtract the middle, and it's the end.
In fewer steps than the usual way,
Karatsuba keeps time at bay.



Karatsuba's Algorithm: A Rhyme

The work it does is sharp and keen,
 $O(n^{\log_2 3})$ —it's faster, lean.
Instead of n^2 , slow and grim,
Karatsuba's pace stays trim.

So when the numbers grow too big,
This algorithm will do the gig.
It multiplies with speed and grace,
Karatsuba wins the race!

